MÓDULO 6 FRONTEND (20H)

JavaScript e TypeScript no Navegador

Manipulação e Seleção Avançada de Elementos via DOM — Guia Detalhado com Explicação de Código e Impacto Visual

---

Este material consolida os fundamentos, métodos de seleção, manipulação de conteúdo, atributos, ciclo de vida de nós, tipagem estática com TypeScript e tratamento de eventos no navegador. Cada bloco de código original é acompanhado da explicação detalhada de suas instruções e do reflexo visual produzido na interface renderizada pelo browser.

1. CONCEITO FUNDAMENTAL

O que é o DOM (Document Object Model)?

Quando o navegador carrega uma página HTML, ele converte cada etiqueta (tag) em um objeto vivo na memória, organizando-os em uma estrutura hierárquica conhecida como **Árvore do DOM**. Essa arquitetura se apoia em três pilares essenciais:

Pilar Conceitual	Definição Técnica	Impacto Prático no Desenvolvimento
Interface em Árvore	Representação estruturada do documento HTML como nós (`Node`) e elementos (`Element`) legíveis por scripts.	Permite navegar entre elementos pai, filhos e irmãos seguindo a hierarquia `document → html → head / body → header / main / footer`.
Ponte Interativa	Conecta a linguagem de programação (JavaScript / TypeScript) ao layout exibido na tela pelo motor de renderização.	O objeto global `document` funciona como ponto de entrada oficial para consultar ou modificar qualquer parte da página.
Dinamismo Total	Permite alterar conteúdos, estilos, atributos e reagir a eventos em tempo real.	A interface responde às ações do usuário instantaneamente, sem necessidade de recarregar o documento inteiro no servidor.

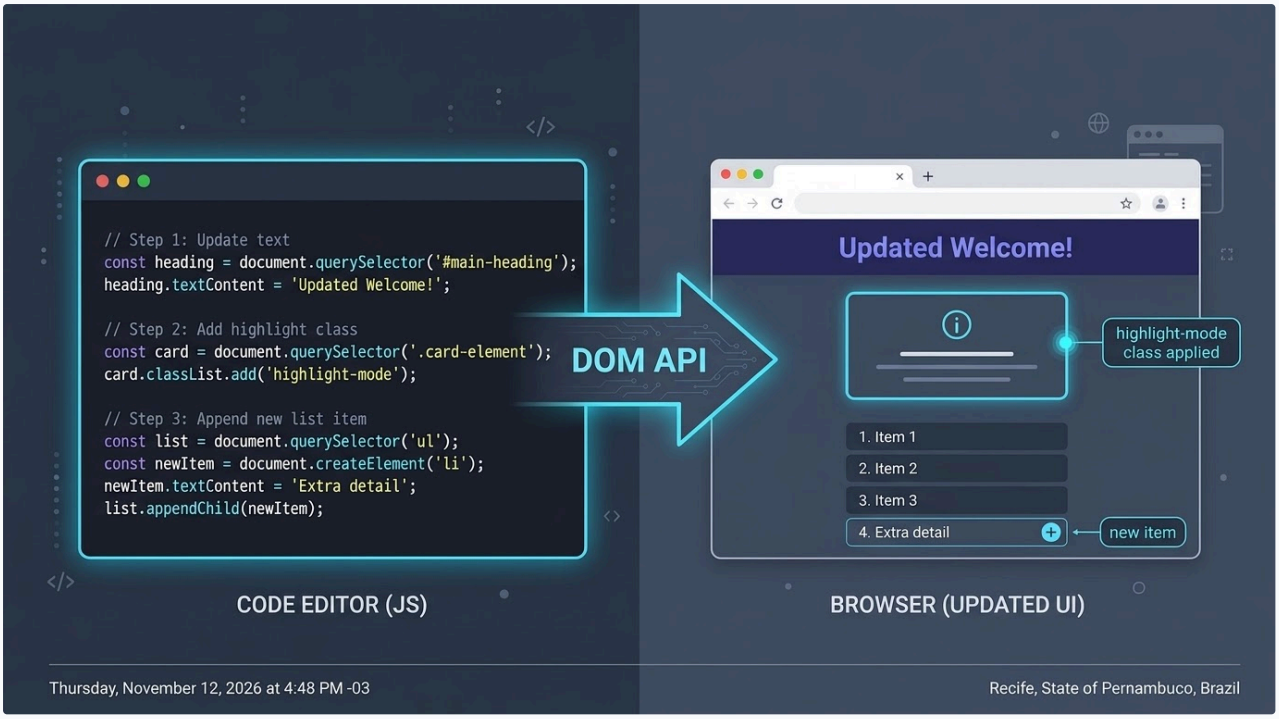


Figura 1 — Fluxo de Atualização via DOM API: À esquerda, instruções em JavaScript alteram o texto de um título (`textContent`), aplicam uma classe visual a um cartão (`classList.add`) e anexam um novo item à lista (`appendChild`). À direita, o navegador reflete imediatamente essas três mutações na interface gráfica.

2. SELEÇÃO DE NÓS

Seleção de Elementos Modernos

Antes de modificar qualquer componente na tela, é indispensável localizar sua referência na árvore do DOM. O código abaixo demonstra as principais formas de captura:

```
// Seleciona o primeiro elemento compatível com o seletor CSS
const titulo = document.querySelector('#titulo-principal');
const primeiroBotao = document.querySelector('.btn-primary');

// Seleciona TODOS os elementos compatíveis (Retorna uma NodeList)
const todosItens = document.querySelectorAll('.lista-item');

// Métodos legados/específicos (HTMLCollection)
const container = document.getElementById('app');
```

Trecho de Código	Explicação Técnica e Comportamento
`document.querySelector('#titulo-principal')`	Recebe qualquer seletor CSS válido (neste caso, o ID `#titulo-principal`) e percorre o DOM de cima para baixo, retornando **apenas o primeiro elemento** encontrado ou `null` caso não exista.
`document.querySelector('.btn-primary')`	Mesmo que existam vários botões com a classe `.btn-primary` na página, `querySelector` interrompe a busca na primeira ocorrência e devolve somente o primeiro botão.

Trecho de Código	Explicação Técnica e Comportamento
`document.querySelectorAll('.lista-item')`	Varre todo o documento e retorna uma coleção estática do tipo `NodeList` contendo **todos** os elementos com a classe `.lista-item`. Permite iterar diretamente com `.forEach()`.
`document.getElementById('app')`	Método clássico otimizado exclusivamente para busca por atributo `id` (dispensa o símbolo `#`). Métodos irmãos como `getElementsByClassName` retornam uma `HTMLCollection` dinâmica (atualizada automaticamente quando o DOM muda).

3. MANIPULAÇÃO

Alterando Conteúdo e Texto

Com a referência do nó armazenada em uma variável, podemos ler ou sobrescrever o conteúdo exibido ao usuário utilizando três propriedades fundamentais:

Propriedade	Exemplo de Código	Explicação e Efeito no Browser
`textContent`	`elem.textContent = 'Olá Mundo!';`	Altera exclusivamente o **texto puro** interno do elemento. Qualquer tag escrita na string é exibida literalmente como texto na tela, tornando este método **seguro contra ataques XSS** (Cross-Site Scripting).
`innerHTML`	`elem.innerHTML = 'Texto';`	Injeta e interpreta **marcação HTML interna**, fazendo o navegador criar novos nós filhos na árvore. Exige sanitização rigorosa caso o conteúdo venha de entradas de usuários para evitar execução de scripts maliciosos.
`value`	`input.value = 'novo@email.com';`	Diferente de parágrafos ou títulos, campos de formulário (`<input>` , `<select>` , `<textarea>`) não guardam dados entre tags de abertura e fechamento; `.value` lê ou preenche diretamente o texto visível dentro da caixa de entrada no navegador.

4. ATRIBUTOS & ESTILOS

Classes CSS e Atributos HTML

Para alterar a aparência visual e o comportamento funcional de elementos sem misturar regras de estilo diretamente no JavaScript, manipulamos suas classes CSS e atributos HTML:

```
// Manipulação de Classes com classList
card.classList.(, );
card.classList.();
card.classList.();


imagem.(, );
 url = imagem.();
link.();add'ativo''destaque'remove'inativo'toggle'dark-mode'// Alterna
liga/desliga// Manipulação direta de AtributossetAttribute'src''foto-
perfil.jpg'constgetAttribute'src'removeAttribute'disabled'
```

Instrução	O que acontece na árvore e na tela
`card.classList.add('ativo', 'destaque')`	Adiciona múltiplas classes CSS de uma só vez ao elemento `card`. O navegador recalcula os estilos e aplica imediatamente as regras definidas para `.ativo` e `.destaque`.
`card.classList.remove('inativo')`	Retira a classe `inativo` do atributo `class` do elemento, removendo sua estilização associada.
`card.classList.toggle('dark-mode')`	Funciona como um interruptor: se a classe `dark-mode` não estiver presente, ela é adicionada; se já existir, é removida. Muito usado em botões de alternância de tema.
`imagem.setAttribute('src', 'foto-perfil.jpg')`	Define ou substitui o atributo `src` da tag `<img>`, fazendo o navegador disparar o carregamento e exibir a nova imagem na página.
`imagem.getAttribute('src')`	Lê o valor textual atual armazenado no atributo `src` e o salva na constante `url`.
`link.removeAttribute('disabled')`	Remove completamente o atributo booleano `disabled`, reabilitando a interação e o clique do usuário sobre o elemento.

5. CICLO DE VIDA DO NÓ

Criando, Inserindo e Removendo Nós

Além de editar elementos já existentes no HTML inicial, o JavaScript permite construir novos nós em memória, inseri-los na página e eliminá-los dinamicamente:

```
// 1. Criar novo elemento
 novoItem = document.();
novoItem.textContent = ;


 lista = document.();
lista.(novoItem);


novoItem.(); constcreateElement'li''Novo item da lista'// 2. Anexar à árvore do
DOMconstquerySelector'ul'appendChild// Insere no final// 3. Remover do DOMremove//
Remove diretamente
```

- **Etapa 1 — Instanciação em Memória (`createElement` e `textContent`):** `document.createElement('li')` fabrica um nó `<li>` isolado na memória RAM (ainda invisível no navegador). Em seguida, `novoItem.textContent` preenche seu rótulo textual antes da inserção na tela.
- **Etapa 2 — Anexação à Árvore (`querySelector` e `appendChild`):** Localizamos o elemento pai `<ul>` e invocamos `lista.appendChild(novoItem)`, que acopla o novo `<li>` como último filho da lista, tornando-o imediatamente visível no navegador.
- **Etapa 3 — Remoção Direta (`remove`):** O método moderno `novoItem.remove()` desvincula o próprio nó da árvore do DOM sem precisar referenciar seu elemento pai, eliminando-o da renderização.

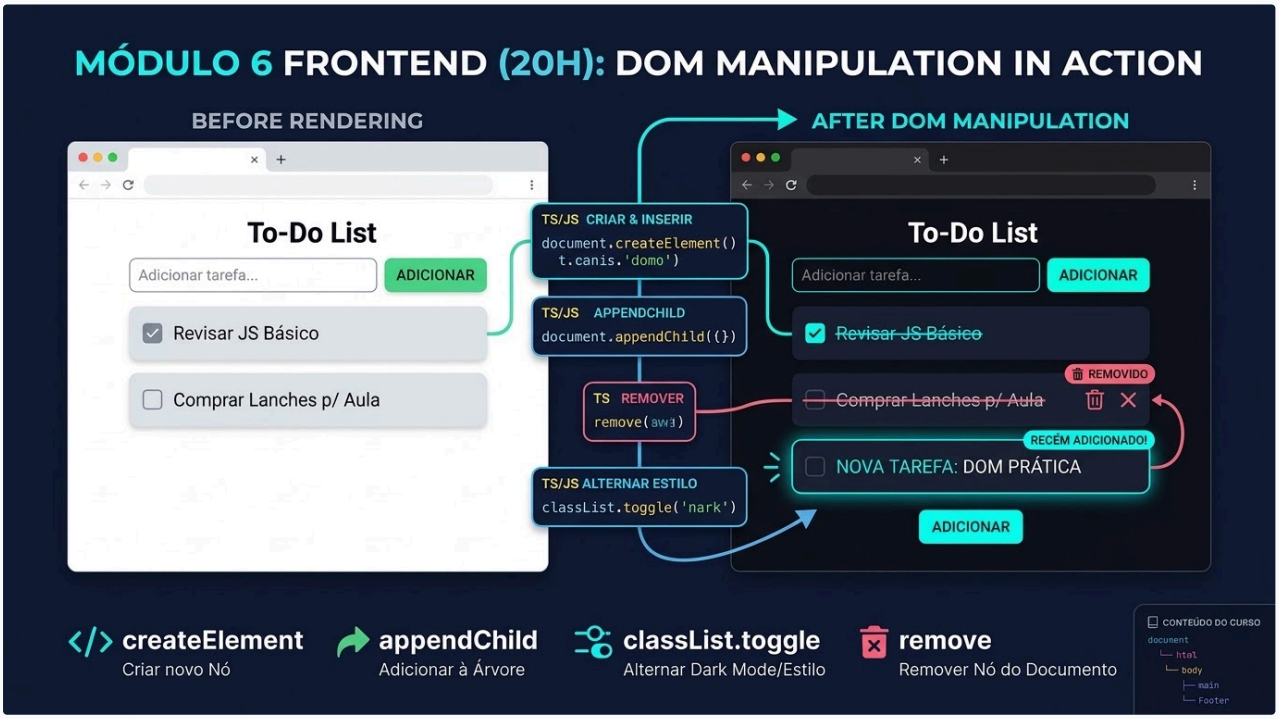


Figura 2 — Ciclo Completo de Mutação no Navegador: Comparação entre o estado inicial da página (esquerda) e o estado atualizado (direita) após a execução combinada de `createElement()` e `appendChild()` (inserção de nova tarefa), `remove()` (exclusão de item) e `classList.toggle()` (alternância para modo escuro).

6. TIPAGEM ESTÁTICA

DOM no TypeScript: Type Casting & Null Checks

Em TypeScript, métodos genéricos como `querySelector` retornam `Element | null`. Como a interface genérica `Element` não possui propriedades específicas de formulários (como `.value`), precisamos informar o tipo exato do elemento e proteger o código contra referências nulas:

```
// Type Casting explícito para acesso a propriedades específicas
const inputName = document.querySelector('#campo-nome') as HTMLInputElement;
console.log(inputName.value);

// Encadeamento Opcional (Optional Chaining) para prevenção de erros nulos
const botao = document.querySelector('.btn');
botao?.classList.add('ativo');
```

Mecanismo TypeScript	Explicação Técnica
`as HTMLInputElement` (Type Casting)	Garante ao compilador que `#campo-nome` é um campo de entrada HTML, liberando o autocompletar e a leitura segura de `inputName.value` sem erro de compilação.
`botao?.classList.add('ativo')` (Optional Chaining)	O operador `?.` verifica se `botao` é diferente de `null` ou `undefined` antes de acessar `classList`. Se o seletor não encontrar o botão no HTML, a execução ignora a linha silenciosamente em vez de lançar uma exceção `TypeError` em tempo de execução.

7. INTERATIVIDADE

Escutando Eventos (EventListeners)

Os ouvintes de eventos conectam ações físicas do usuário (cliques, digitação, envio de formulários) à execução de funções JavaScript:

```
const btnEnviar = document.querySelector('#btn-enviar');

btnEnviar?.addEventListener('click', (event) => {
  event.preventDefault(); // Impede o comportamento padrão
  console.log('Formulário enviado com sucesso!');
});
```

- **`btnEnviar?.addEventListener('click', ...)`**: Registra uma função de retorno (callback) que fica aguardando o evento de clique (`'click'`) no botão `#btn-enviar`, protegida pelo encadeamento opcional `?.`.
- **`event.preventDefault()`**: Bloqueia a ação nativa do navegador associada ao elemento (como recarregar a página ao submeter um botão dentro de um `<form>`), permitindo validar e processar os dados via JavaScript sem perda de estado na tela.
- **`console.log(...)`**: Registra a confirmação no console das ferramentas de desenvolvedor do navegador após interceptar o evento.

8. PRÁTICA GUIADA

Exercício: Lista Dinâmica de Tarefas (To-Do)

A integração de todos os conceitos anteriores materializa-se no fluxo de quatro passos para construir uma lista dinâmica de tarefas interativa:

Passo	Etapa	Implementação via APIs do DOM
1	**Capturar Input**	Selecionar o campo de texto com `querySelector` (tipado como `HTMLInputElement`) e obter o texto digitado pelo usuário através de `input.value`.
2	**Criar Elemento**	Instanciar um novo item de lista `<li>` em memória via `document.createElement('li')` e preencher seu rótulo com `textContent`.
3	**Inserir na Lista**	Anexar o novo nó `<li>` dentro do contêiner `<ul>` usando `lista.appendChild(novoItem)` para exibi-lo imediatamente na tela.
4	**Remover ao Clicar**	Adicionar um escutador `addEventListener('click', ...)` no próprio item (ou em seu botão de exclusão) chamando `item.remove()` para eliminá-lo do DOM.

9. RESUMO DA CARGA HORÁRIA

Boas Práticas na Manipulação do DOM

Diretrizes essenciais para aplicações Frontend robustas:

- **Segurança:** Prefira `textContent` em vez de `innerHTML` ao lidar com dados externos ou fornecidos por usuários, neutralizando vulnerabilidades de injeção de scripts (XSS).
- **Performance:** Evite leituras e escritas excessivas e intercaladas no DOM dentro de loops para prevenir recalculos de layout (*reflows*) e repinturas pesadas no navegador.
- **Tipagem com TS:** Sempre especifique os tipos corretos dos elementos do DOM (como `HTMLInputElement`, `HTMLButtonElement`) e utilize verificações de nulidade para prevenir erros em runtime.